Snowdrift

Project starter sheet · macOS v1 · working title

Snow that falls over your whole desktop, piles up on windows and the Dock, and reacts to your mouse and to windows you move, with physics close to real life. A personal project for my GitHub portfolio, used on my and my friends' Macs. A Windows 11 version comes later.

1. What v1 does

Falling snow

- **Overlay:** snow falls over every app, on every screen. Clicks and typing pass through it.
- **Flakes:** soft round dots, 2–9 px, each with its own size, opacity, speed and sideways sway. The biggest are slightly blurred for depth.
- **Density:** calm and light by default, adjustable from the menu bar.

Snow that piles up

- **Surfaces:** the top edge of every visible window, the Dock, and the bottom of each screen.
- **Shape:** piles grow where flakes land and slump into natural slopes. A pile has a maximum height, so extra snow slides off the ends.
- **Occlusion:** snow on a window that sits behind another window is hidden where the front window covers it.

Mouse interaction

- **Pushing:** moving the cursor through a pile pushes the snow left or right.
- **Compacting:** pushed snow gets denser and a little taller in front of the cursor.
- **Overflow:** the extra amount falls off as small clumps.

Window motion

- **Window moves down:** snow lifts off, the flakes spread apart as they fall, and they land again on any surface below (that window, another window, the Dock or the screen bottom).
- **Window moves up:** snow rides along with the window.
- **Window stops after moving up:** the snow keeps its momentum, gets thrown a little upward, then falls and settles again.
- **Window moves sideways:** snow mostly stays on. A sharp stop makes some of it slide off the ends.
- **Window closes, minimizes or shrinks:** the snow on it, or on the cut-off part, falls.

Controls

- **Menu bar:** a snowflake icon to turn snow on or off. The menu also has density, wind and "clear all snow".
- **Launch at login:** optional.
- **Hide from screen sharing:** optional. Snow then doesn't show in Zoom, Meet or screen recordings.

2. Technology

Part	Choice	Why
Language	Swift 6	Native on macOS, fast, direct access to every system API.
App shell	AppKit + SwiftUI MenuBarExtra	AppKit for the transparent overlay windows, SwiftUI for the small menu and settings.
Overlay window	Borderless NSPanel per screen	Transparent, ignoresMouseEvents, above all windows, joins all Spaces and full-screen apps.
Rendering	Metal (MTKView)	Two GPU draw calls per frame: all flakes as instanced sprites, all piles as triangle strips. Lowest CPU and battery cost.
Physics	Own code in Swift (SIMD)	Flakes are simple particles and piles are height maps. A full physics engine would be slower and harder to control.
Window tracking	CGWindowListCopyWindowInfo	Gives position, size, owner and z-order of every window. Bounds need no special permission.
Frame timing	CADisplayLink (macOS 14+)	Syncs simulation, window polling and drawing to the screen's refresh rate.
Mouse	NSEvent.mouseLocation	Read once per frame. No permission needed.
Tools	Xcode 16+, Git, GitHub	Optional later: GitHub Actions to build a release .zip.
Minimum macOS	macOS 14 Sonoma	Same as Koi. Apple Silicon and Intel.

Later Windows port: keep the physics module free of any Apple framework. It then translates almost line by line to C++ or C#. On Windows, the overlay is a layered, click-through, topmost window, window moves are tracked with SetWinEventHook, and rendering uses Direct3D 11.

3. How it works

- **Frame loop, 60 times a second:** read window positions → read the mouse → step the physics → draw.
- **Window tracking:** for every window, keep its position over the last few frames to get its velocity and acceleration.
- **Flakes:** each one has a position and a velocity. Gravity pulls it down to a low top speed (real snow falls at about 1 m/s), and a sine wave plus gentle noise makes it sway.
- **Piles:** each surface stores a column of snow height every 2 px. A landing flake adds height to its column.
- **Slumping:** each frame, a column that is much taller than its neighbour passes some snow across (a sand-pile rule). Snow that reaches the end of a surface falls off as a clump.
- **Coordinates:** piles are stored relative to their window, so they move with it for free.
- **Detaching:** if a window accelerates downward faster than gravity, its snow turns back into falling particles. They start with the window's speed plus a small random spread, so they drift apart.

- **Thrown upward:** if a window moving up decelerates, its snow is launched with the window's last velocity, flies a little higher, then lands again.
- **Mouse push:** inside a pile, the cursor moves column height in its direction of travel. Height is pushed to the front and the pile gets a bit taller there. Anything above the maximum or past an end becomes falling clumps.
- **Limits:** a cap on flakes and pile height plus a slow melt, so memory and CPU stay flat however long it runs.

4. Risks and how we handle them

Area	Risk	How we handle it
CPU	Physics and window polling every frame.	Under 1,500 flakes, height maps instead of rigid bodies, window polling at 30 Hz with smoothing in between. Goal: under 3% of one core.
GPU / WindowServer	A full-screen transparent window redrawn 60 times a second costs the system compositor work, even when nothing moves.	Metal with tiny draw calls, 60 fps cap on 120 Hz screens, lower frame rate once snow has settled.
Battery	Constant drawing keeps the GPU awake.	Pause when the screen is locked or asleep, a screensaver is running, or Low Power Mode is on. Off by default on battery (setting).
Memory	Piles grow forever; leaked windows or textures.	Fixed-size buffers allocated once and reused. Piles capped and melting. Goal: under 80 MB.
Lag behind windows	Our overlay draws a frame or two after the real window moves, so snow can trail a dragged window.	Poll on the display link, predict one frame ahead from the window's velocity. Main technical challenge; prototype it first.
Blocking clicks	A bug that makes the overlay catch mouse input would make the Mac hard to use.	ignoresMouseEvents is always on and the overlay has no interactive parts. Global quit shortcut plus a menu bar item.
Privacy	Screen Recording permission would let the app see screen contents.	Not needed or requested. Only window bounds are read (no titles, no pixels). No network access at all.
Security	Unsigned app; tampering or fake copies.	Hardened runtime, Developer ID signing and notarization if I get a paid Apple account. Otherwise friends approve it once in System Settings → Privacy & Security → Open Anyway.
Screen sharing	Snow shows up in calls and recordings.	Option to set sharingType = .none on the overlay.
Edge cases	Full-screen apps, Mission Control, multiple monitors, Stage Manager, menu bar and notch.	One overlay per screen, collectionBehavior for all Spaces, re-check screens when monitors connect or disconnect.

5. Targets

Metric	Target
CPU (Activity Monitor)	< 3% average while snowing, < 1% when settled
Memory	< 80 MB
Frame rate	60 fps, no visible stutter while dragging windows
Permissions	None (no Accessibility, no Screen Recording)
Network	None
App size	< 10 MB

6. Milestones

- **M0 – Spike:** one transparent, click-through overlay with Metal drawing falling flakes. Measure CPU.
- **M1 – Window tracking:** list windows each frame and draw their outlines for debugging. Measure lag while dragging.
- **M2 – Piles:** snow settles on window tops, the Dock and the screen bottom. Slumping, overflow and occlusion.
- **M3 – Motion physics:** riding along, detaching, throw-up and resettling.
- **M4 – Mouse push:** compacting and overflow.
- **M5 – Polish:** menu bar controls, multiple monitors, pause rules, settings, launch at login.
- **M6 – Ship:** signing, README with GIFs, GitHub release.

7. Open decisions

- **Name:** “Snowdrift” is a placeholder.
- Apple Developer account ($99 a year) for notarization, or friends use “Open Anyway”.
- **Windows port later:** rewrite the physics in C#/C++, or move it into a shared Rust or C++ core from the start.